Proyecto RDA1: Detección de Phishing en URLs con Machine Learning

Alejandro Flores, Paúl Rosero y Gloria Chushig

Inteligencia Artificial

11 de octubre de 2026

1. Introducción

El phishing es uno de los ataques informáticos más frecuentes: el atacante envía un enlace que imita a una entidad legítima (un banco, una red social, un servicio de pagos) para robar credenciales o datos financieros.

Este proyecto desarrolla **PhishGuard**, un sistema que clasifica una URL como **benigna (0)** o **phishing (1)** a partir únicamente de sus características léxicas y estructurales, sin visitar la página. Se entrenaron y compararon cuatro modelos supervisados de scikit-learn (Regresión Logística, SVM lineal, SVM con kernel polinómico y SVM con kernel gaussiano) y el ganador se desplegó en una aplicación web con Streamlit que entrega la probabilidad de phishing de 0 a 100 %.

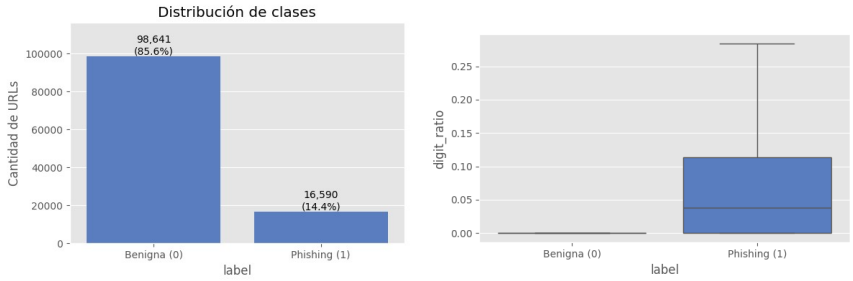
2. Datos y análisis exploratorio

Se utilizó el dataset *Phishing URL Detection (111K URLs, 22 Features)* de Kaggle: 116.600 URLs con 22 variables numéricas ya calculadas (longitudes, conteos de caracteres, proporciones, entropía, uso de HTTPS e IP) y la etiqueta `label`. Durante la limpieza se tomaron dos decisiones:

- **Nulos:** solo 14 valores faltantes en `tld`, todos de URLs que son una IP pura (no tienen dominio de nivel superior). Se imputaron como `none`; `tld` es texto y no entra al modelo.
- **Duplicados:** 1.369 URLs repetidas, todas con la misma etiqueta. Se eliminaron para que una misma URL no aparezca en entrenamiento y prueba a la vez (fuga de datos). Quedaron **115.231 URLs únicas**.

La clase está **desbalanceada** (85,6 % benignas y 14,4 % phishing), lo que refleja la realidad de la web. Por eso la división 80/20 se hizo con `stratify=y`, los modelos usan `class_weight="balanced"` y la evaluación prioriza Recall, Precision, F1 y AUC de la clase phishing en lugar del *Accuracy*. Las variables más correlacionadas con phishing son `digit_ratio` (r = 0,52), `entropy` (0,51) e `is_https` (0,40): el 91 % de las URLs de phishing usa HTTPS frente al 35 % de las benignas, por lo que el candado **no** es garantía de legitimidad.

Figura 1
Distribución de clases (izquierda) y proporción de dígitos por clase (derecha)

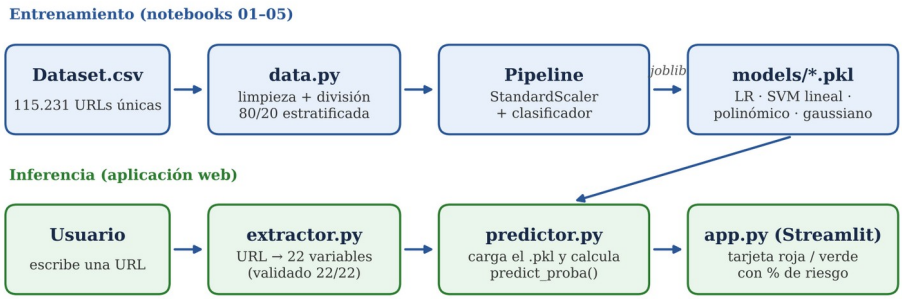


3. Arquitectura de la solución

El código está organizado en módulos compartidos (`src/`) que usan por igual los notebooks, los scripts y la aplicación, de modo que los cuatro modelos se entrenan con **la misma partición y el mismo preprocesamiento**. Cada modelo es un `Pipeline` de scikit-learn (un `ColumnTransformer` con `StandardScaler`, necesario porque los cuatro modelos son sensibles a la escala, seguido del clasificador), lo que evita fuga de información al escalar. El pipeline se exporta con `joblib` y la app lo carga para inferencia.

La pieza crítica es `extractor.py`, que convierte la URL escrita por el usuario en las mismas 22 variables del dataset. Se validó comparando sus cálculos con los valores del dataset en 2.000 URLs al azar: **22 de 22 variables coinciden exactamente**. Para separar dominio, sufijo y subdominios (p. ej. `espe.edu.ec`) se usa la librería `tldextract`.

Figura 2
Arquitectura de PhishGuard: entrenamiento e inferencia



4. Modelado, comparación y selección del modelo

Se entrenaron cuatro modelos con 92.184 URLs (80 %) y se evaluaron con 23.047 URLs de prueba (20 %, de las cuales 3.318 son phishing). La Tabla 1 resume sus configuraciones y la Tabla 2 sus resultados.

Tabla 1
Configuración de los modelos

Modelo (responsable)	Configuración principal	Probabilidad
Regresión Logística (Paúl Rosero)	`lbfgs`, `max_iter=1000`, `class_weight="balanced"`	Sigmoide

Modelo (responsable)	Configuración principal	Probabilidad
SVM lineal (Gloria Chushig)	`LinearSVC(C=1.0, class_weight="balanced")` calibrado (sigmoide, 5 particiones)	Calibración
SVM polinómico (Alejandro Flores)	`SVC(kernel="poly", degree=3, coef0=1, C=1.0, class_weight="balanced")`	Platt
SVM gaussiano (Alejandro Flores)	`SVC(kernel="rbf", gamma="scale", C=1.0, class_weight="balanced")`	Platt

Tabla 2
Resultados en el conjunto de prueba (clase phishing)

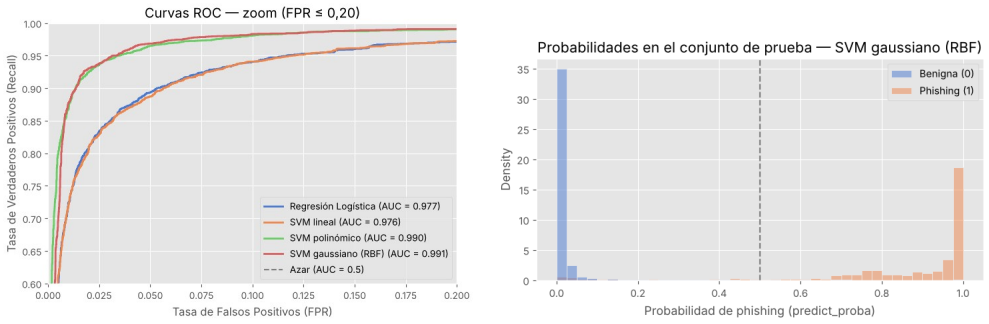
Modelo	Recall	Precision	F1	AUC	Accuracy	FN	FP
Regresión Logística	**0,920**	0,686	0,786	0,977	0,928	**264**	1.396
SVM lineal	0,758	0,910	0,827	0,976	0,954	803	248
SVM polinómico	0,862	**0,940**	0,899	0,990	0,972	459	**182**
SVM gaussiano	0,903	0,915	**0,909**	**0,991**	**0,974**	322	277

Nota. FN = ataques no detectados; FP = falsas alarmas. En negrita, el mejor valor de cada columna.

Para comprobar que la diferencia no depende de una partición afortunada se aplicó **validación cruzada estratificada de 5 particiones** sobre el conjunto de entrenamiento. Los resultados fueron estables (desviación ≤ 0,008) y se mantuvo el mismo orden: el SVM gaussiano obtuvo el mejor F1 (0,888 ± 0,003) y AUC (0,989 ± 0,001), seguido del polinómico (F1 0,879), el SVM lineal (0,821) y la Regresión Logística (0,784).

La Figura 3 muestra dos grupos de curvas ROC. Los modelos **lineales** (Regresión Logística y SVM lineal) son prácticamente idénticos (AUC 0,977 y 0,976): sus diferencias de Recall y Precision se deben al punto de corte, no a su capacidad de separar las clases. Los SVM con **kernel** están claramente por encima (AUC 0,990 y 0,991): con solo 2,5 % de falsas alarmas detectan ≈ 93–94 % del phishing, porque el *kernel trick* permite fronteras no lineales que combinan variables (p. ej. HTTPS con alta entropía y muchos dígitos).

Figura 3
Curvas ROC con zoom en FPR ≤ 0,20 (izquierda) y probabilidades del SVM gaussiano por clase real (derecha)



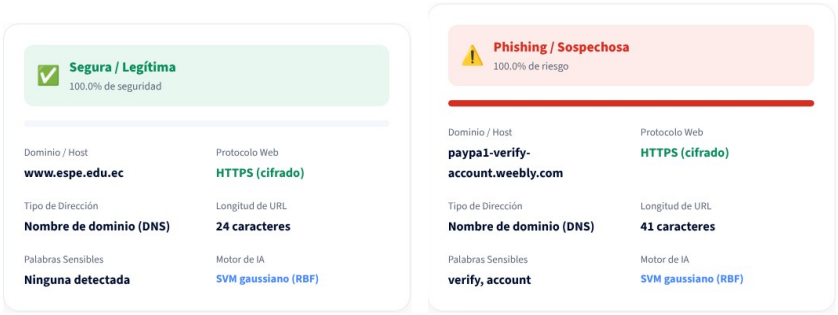
Modelo seleccionado: SVM gaussiano (RBF). Tiene el mejor F1 (0,909) y AUC (0,991), confirmados con validación cruzada. En ciberseguridad un falso negativo es una

posible víctima, por lo que el Recall es prioritario: la Regresión Logística detecta algo más (92,0 % frente a 90,3 %), pero genera **5 veces más falsas alarmas** (1.396 frente a 277), y un detector que se equivoca en 1 de cada 3 alertas termina siendo ignorado. Frente al polinómico, el gaussiano detecta 137 ataques más a cambio de solo 95 falsas alarmas adicionales. Su costo es un entrenamiento más lento (≈ 6 min) y menor interpretabilidad. Se reentrenó con el 100 % de los datos (`models/modelo_svm_rbf.pkl`).

5. Aplicación web

La aplicación en Streamlit tiene tres páginas: **Analizar URL**, **Modelos IA** (con la tabla de métricas) y **Ayuda y consejos**. El usuario pega una URL, el extractor calcula las 22 variables, el SVM gaussiano devuelve la probabilidad y la app muestra una tarjeta roja con el nivel de riesgo o una verde de sitio seguro, junto con el host, el protocolo, el tipo de dirección y las palabras sensibles detectadas. Un selector permite comparar la misma URL con los cuatro modelos.

Figura 4
Resultados de la aplicación con el SVM gaussiano: URL legítima (izquierda) y URL de phishing (derecha)



Limitaciones. El análisis es léxico: no visita la página ni revisa su contenido. Además, el dataset contiene sobre todo **páginas principales** de sitios legítimos, por lo que un enlace legítimo profundo (una búsqueda, un artículo, un repositorio) puede recibir un riesgo mayor al real; y solo 13 URLs del dataset son una IP, así que el modelo no aprendió a usar esa señal. Ambas limitaciones se informan en la propia aplicación.

6. Instalación y ejecución local

Requisitos: Python 3.11 o superior. Desde la raíz del proyecto:

```
git clone https://github.com/alex171215/Detector-Phishing-ML.git
cd Detector-Phishing-ML
pip install -r requirements.txt
python -m streamlit run app.py
```

La app se abre en `http://localhost:8501`. Los modelos ya entrenados están en `models/`; para reentrenarlos se descarga `Dataset.csv` de Kaggle en `data/` y se ejecuta `python -m scripts.entrenar_todos`. La versión `scikit-learn==1.9.1` está fijada en `requirements.txt` para que los `.pkl` carguen igual en cualquier equipo.

7. Conclusiones

- El **SVM gaussiano** fue el mejor modelo (F1 0,909, AUC 0,991, Recall 90,3 %, Precision 91,5 %), con ventajas estables en validación cruzada.
- Los **kernels** marcan la diferencia: con el mismo `C = 1`, pasar del SVM lineal al polinómico o al gaussiano sube el AUC de 0,976 a 0,990–0,991, porque las fronteras no lineales captan combinaciones de variables que un hiperplano no puede representar.
- El éxito en producción depende tanto del modelo como del **extractor**: validarlo variable por variable contra el dataset fue indispensable para que la app prediga correctamente.
- Como trabajo futuro se propone ampliar el dataset con enlaces profundos legítimos y URLs con IP, y ajustar `C`, `gamma` y el umbral de decisión con validación cruzada según el costo de cada tipo de error.

Referencias

Namvar, S. (s.f.). *Phishing URL Detection (111K URLs, 22 Features)* [Conjunto de datos]. Kaggle. https://www.kaggle.com/datasets/sahandnamvar/phishing-url-detection-111k-urls-22-features

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., … Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.